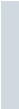


01. OOP and Software Design



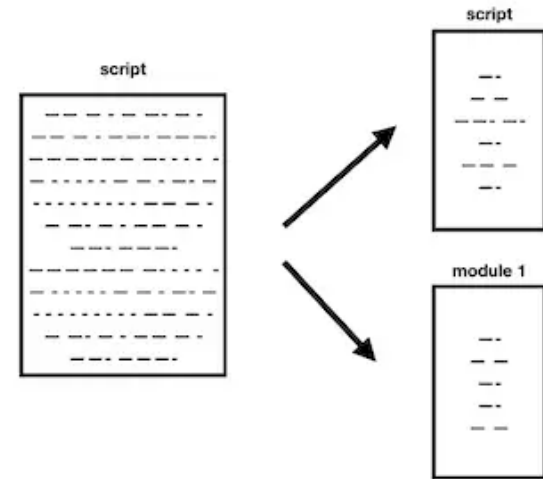
Two Questions

1. Why OOP?

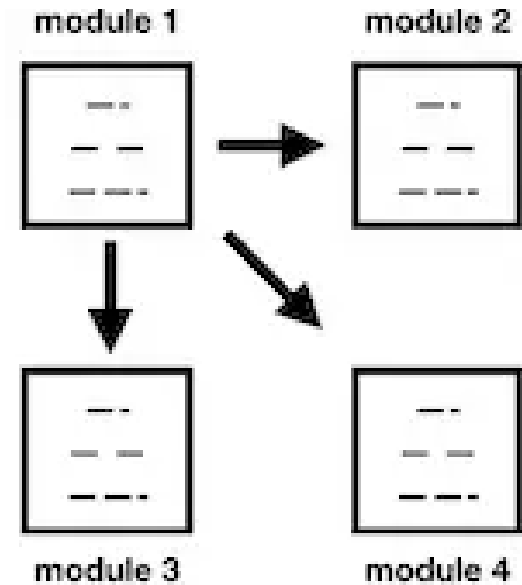
2. Why Software Design?

Starting Small

- When we start programming, we start small.
- We write a small script to solve the given problem.



- However, the script becomes hard to read and manage.
- So we split the script into smaller parts.

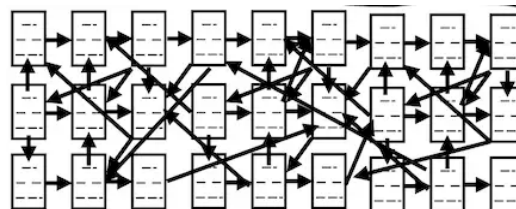


Growing Complexity

- As we develop software, we add more and more modules.



- Also, the couplings and dependencies among modules increase over time.



- Very quickly, we find that managing the modules becomes costly.
- We cannot respond to users' requests.
- We introduce more bugs when we fix bugs.
- We pay the price of **ad-hoc development** (development without design).

The Problems - Changes and Complexity

- Everything changes in programming because:
 - Clients ask for new features
 - We adopt new tools
 - We update libraries
 - and so on...

Code always grows
Code always changes

- The changes add more and more complexity to programming.
- When we focus only on making code, we can never manage the complexity of the changes.

Real-World Software is Inherently Complex

- Software is the most complex system that humans have ever experienced.
- **Managing complexity** is the core of successful software development.

Challenge: How do we manage the complexity?

OOP as a Rescue

- OOP provides the tools and rules for managing complexity.
- OOP is the best way to manage complexity in commercial software.
- However, OOP may not fit every program — small scripts can find it overkill.
- Still, without OOP, it is not possible to build complex modern software.

Abstraction

- In the real world, we perceive anything as an object, not a combination of parts.
- We call this idea **abstraction**.
- OOP's class/object is a smart way to manage modules and interfaces.
- We think of a car as a single entity, not as thousands of individual parts.

Polymorphism

- OOP allows us to use the same method name for different objects.
- We call this idea **polymorphism**.
- Imagine you ask to "speak":
 - Sarah (human student): She says, "Hello, my name is Sarah."
 - Rover (class pet dog): He barks "Woof! Woof!"
- They use the same command ("speak"), but

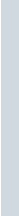
Inheritance

- OOP's **inheritance** allows us to avoid making any objects from scratch.
- Instead, we can modify (override) or extend existing features.

- All vehicles share common features (engine, wheels, steering, brakes).
 - Sports Car: Inherits base vehicle features + adds turbo engine, racing tires
 - Truck: Inherits base vehicle features + adds cargo bed, towing capacity

Encapsulation

- OOP protects data from the outside world using **methods** to access it.
- We manage complexity by limiting changes from the outside.
- We call this idea **encapsulation**.
- Similarly, pharmaceutical companies encapsulate pills to protect the medicine until it reaches our stomach.



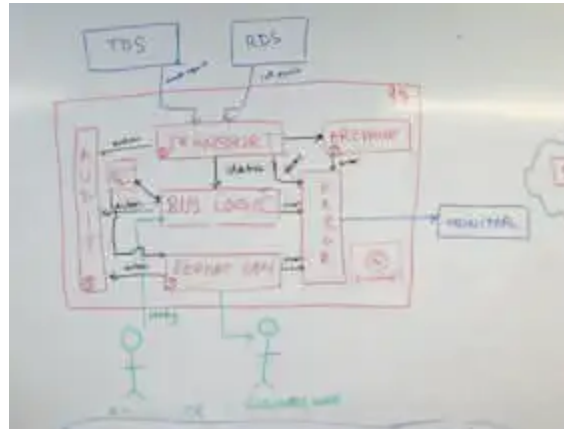
Challenge: How do we manage the complexity?

- We can manage complexity using the OOP/APIE principles.
 - In our daily lives, we use APIE-like principles to simplify our lives.
 - Likewise, we use APIE to manage complexity.

Challenge: How do we sense and prevent the complexity?

Software Design

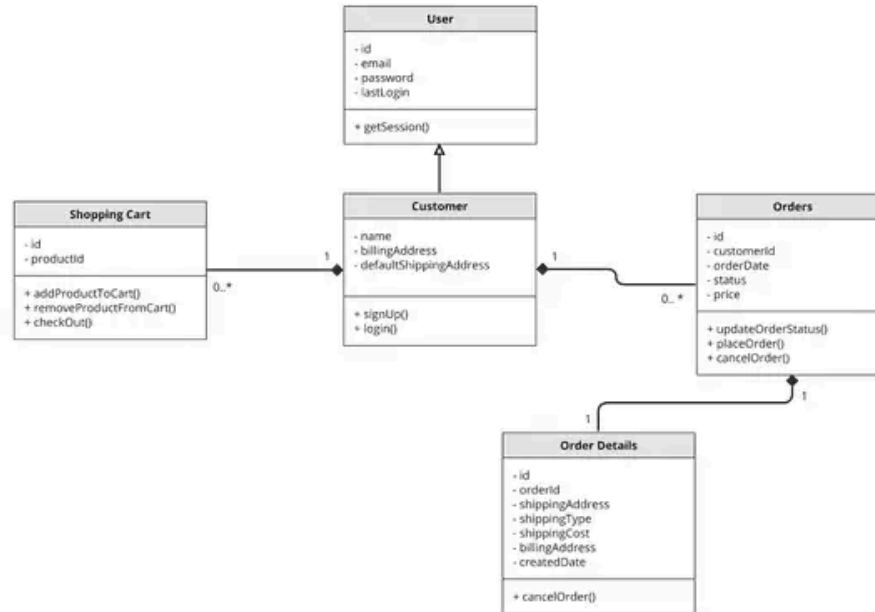
- Software design is about **modules** and **interfaces** to manage complexity.
- In other words, software design means designing modules and their interfaces.



- That is the reason why we use diagrams to describe software design.

UML as a Design Language

- Software engineers developed a unified diagram to express modules and interfaces.
- It is called **UML** - Unified Modeling Language.



- The **three-section box** indicates a module.
- The **methods/fields with the +/-/# sign** indicate interfaces.

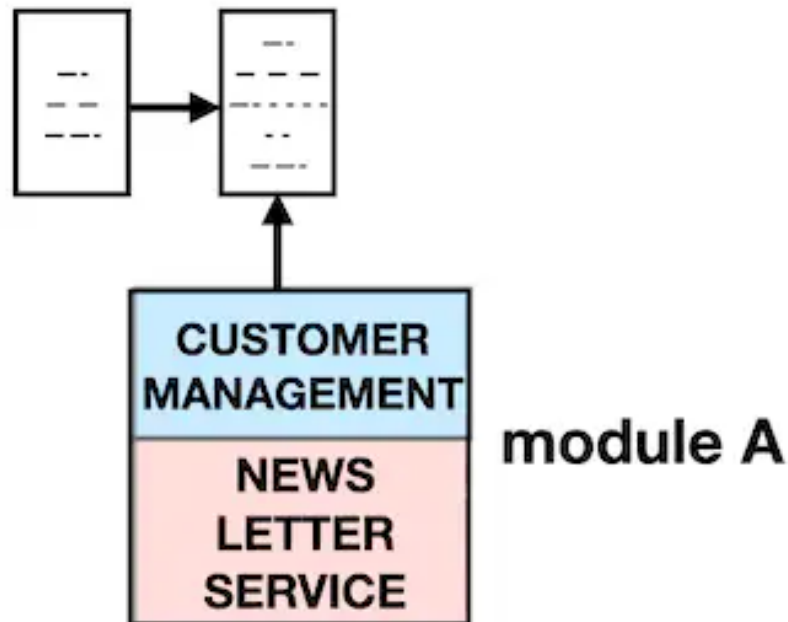
OOP as a Tool for Software Design

- **OOP classes** are modules.
- **OOP public functions** are interfaces.
- OOP provides the building blocks for good software design.

Making modules and interfaces is not enough

- For example, clients ask us to build a newsletter service.
- We design (create modules with interfaces) to implement a feature.
- However, we can make *bad* design easily.

- We may create one module for both customer **and** newsletter features.



- This is **bad**, as it violates the **SRP** (we will discuss it soon) principle.
- Mixing unrelated responsibilities in one module adds unmanageable complexity.

Real-World Example: Bridge Coupling



- A city renovates a bridge, and workers cut it without knowing it's **coupled** with the power line.

- This simple cutting action blackouts the city.
- This (real) incident happened because the designers violated "**low coupling, high cohesion.**"
- We need software **design rules** and to follow them!

Change is Inevitable, But

Change is dangerous

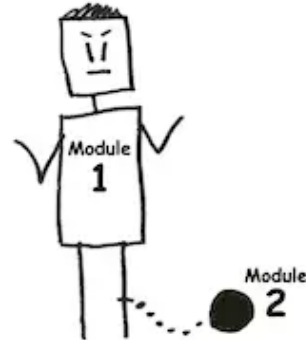
- Change is inevitable, but it's dangerous without a design and design rules.

- We **cannot manage complexity** with ad-hoc design (just modules and interfaces).
- Violating design rules adds complexity, so we can't meet clients' requests.

Managing Complexity from Changes

- We should avoid **rigid code**, which is hard to change.
- We also should avoid **fragile code**, which is easy to break.

Rigid Code



Fragile Code

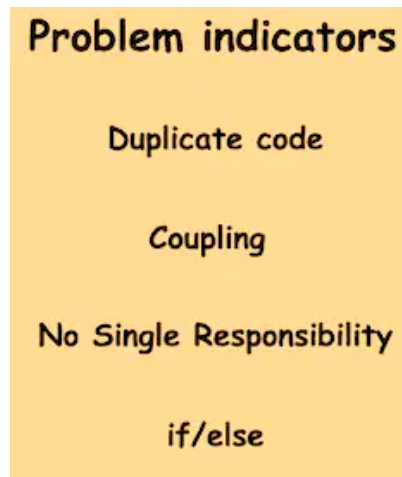


- To avoid rigid and fragile code, we need to follow **software design rules**.
- In other words, what rules help us write code that adapts to change gracefully?

Code smell

- We can sense that something is wrong with the problem indicators.
- Software engineers call that "**code smell**."

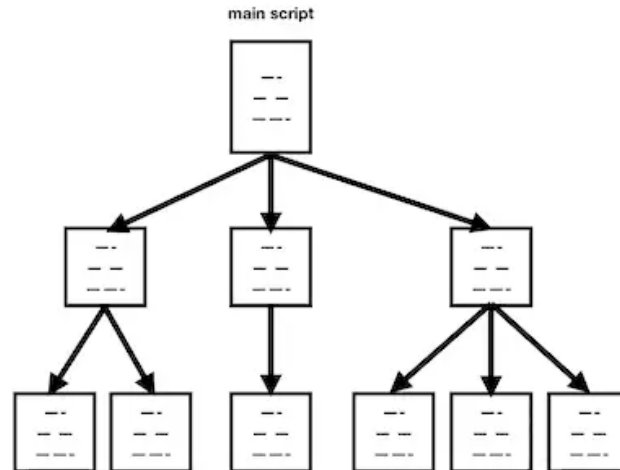
Problem indicators



- We should feel uncomfortable whenever we see these indicators (code smell).

- The problem indicators on the left side are well-known code smells.
- We already know the solutions:
 - Software Design (modules + Interfaces) and the APIE.
 - The APIE rules can guide us in making good modules and interfaces.

Making modules as the first step



- Organizing code into **modules** (and interfaces) is the first design step.

- When code is organized, we can smell and refactor the code smell better.
- In other words, we can manage complexity better using **modularization**.
- However, we need rules to guide the modularization.

Solution toolbox

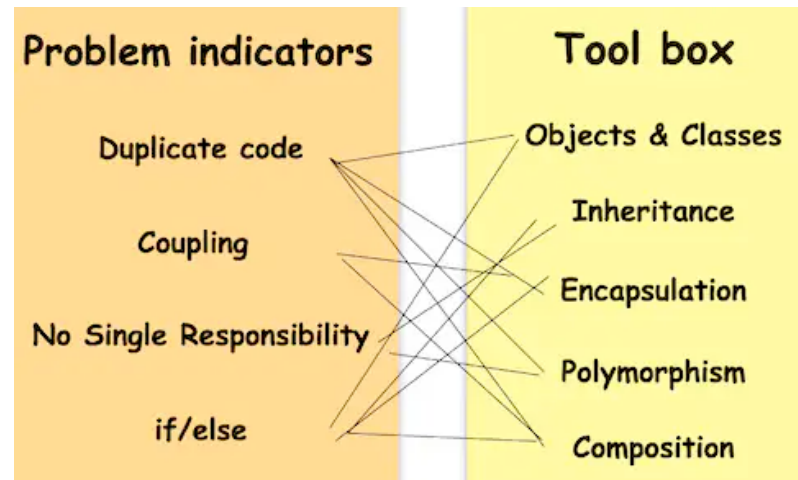


- We have a toolbox to address the issues when we use OOP/APIE.

| APIE + C

- They are **Abstraction** (objects/classes), **Polymorphism**, **Inheritance**, and **Encapsulation** (APIE).
- We have one more tool: **Composition**.

Tool-Problem Connection



The **Toolbox** from OOP and software design addresses these **Problem Indicators** (code smell).

Problem Solver

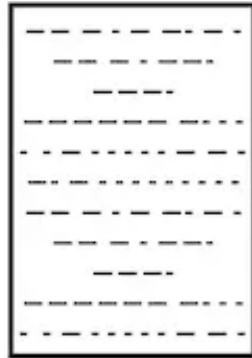
- Good problem solvers/software engineers can sense smelly code.
- They apply software design rules to create better designs and prevent code smells.
- It's an art, and you can work with LLM: You design and AI code/test.

Challenge: How do we sense and prevent the complexity?

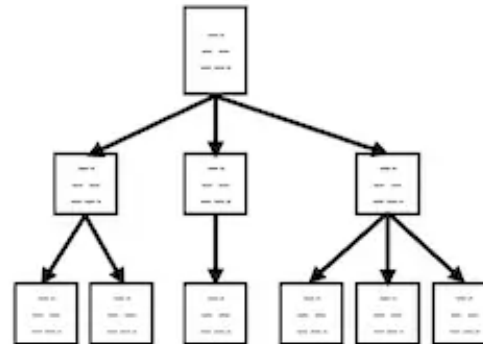
- We identify a set of problem indicators we call `code smell`.
- Software Design removes the code smell guided by OOP/APIEC.

Coder to Designer

Today: Structured programming, one script, runs from top to bottom



After today: OO programs are split into many modules that depend on each other



- We need to transform ourselves from coders to designers.
- It's important, as coders have no chance.